PROJECT SPARK — LEVEL 21 CREATION PROCESS

Capacitor Charge — Store the Energy

Level ID: `level_21`
Purpose: First advanced electronics lesson; build and validate a real charging RC circuit using the existing Spark electrical solver.
Package: `ProjectSpark_Level21_Capacitor_Charge.zip` (contains `SparkLevel21Creator.cs`, reference JSON, README).
Status: Unity authoring guide. The creator generates assets but **does not create a wired playable workbench**.

0. Before starting

- ☐ Back up your project. Work on a new Level 21 prefab, not Level 20.
- ☐ Confirm Unity Console has no compiler errors.
- ☐ Confirm the **active** `SparkElectricalSolverRegistry` has `SparkCapacitorSolverDefinition` (it appeared at Element 8 in your previous screenshot), plus power supply, switch, resistor, and LCD/meter solver definitions.
- ☐ Locate the same working power supply, switch, resistor, LCD, level layout, terminal, and wire prefabs you used in Levels 18–20.
- ☐ Identify the existing capacitor prefab/component and its physical terminal IDs; do not assume a prefab exists merely because the solver definition is installed.

1. Generate Level 21 assets

1. Extract `ProjectSpark_Level21_Capacitor_Charge.zip`.
2. Copy `ProjectSpark_Level21/Editor/SparkLevel21Creator.cs` to an `Assets/.../Editor/` folder.
3. Wait for recompilation and resolve any errors before proceeding.
4. Choose **Tools > Project Spark > Levels > Create Level 21 - Capacitor Charge**.
5. Find the three generated assets in `Assets/ProjectSparkGenerated/Level_21/`:
    - `Level_21.asset` — `SparkLevelDefinition`, three initial targets;
    - `Level_21_Drawing.asset` — native `SparkLevelDrawingAsset`, four pages;
    - `Level_21_Layout.prefab` — intentionally empty layout root with `SparkLevelLayout` and `SparkLevelSceneBindings`.
6. The creator refuses to overwrite existing files. If assets already exist, edit them; do not create duplicate level IDs.

2. Set up the prefab root

1. Open `Level_21_Layout.prefab` in Prefab Mode.
2. Check its `SparkLevelLayout.LevelDefinition` points to `Level_21.asset` and its scene-binding reference points to the prefab's `SparkLevelSceneBindings`.
3. Use the same workbench conventions as Level 20: power supply, switch, resistor workstation, capacitor, LCD, wiring/terminal objects, shelf and slots as needed.
4. **Do not copy Level 20's circuit wires or Level 20 runtime IDs unchanged.** Copy reusable physical prefabs only, then author Level 21 electrical connections.
5. Choose how to mount capacitor: **fixed component (recommended first test)** or a shelf-managed workstation item. Fixed is simpler for debugging. Shelf-managed requires correct `CurrentSlot`, `Occupant`, `InstalledItem`, non-moving/workbench flags, plus runtime ID mapping.

3. Electrical components and values

Object	Initial value / configuration	Role
PowerSupply_01	5.0 V DC	Source
SW1	Initially open, player can close	Charging control
R_CHARGE	1,000 ohms, healthy	Limits charging current
C_STORE	0.001 F = 1000 uF, voltage rating >= 5 V (prefer 16 V), initial voltage ~0 V	Energy storage
LCD	High-impedance voltmeter mode	Reads Vcap

Set any capacitor polarity correctly: capacitor positive goes toward R_CHARGE, negative toward battery negative. Ensure component values are on the actual component used by the solver, not only visual labels.

4. Exact conducting wires (not just visual lines)

Build the **main current path** first:

```
Battery_positive  --> SW1 Input
SW1 Output        --> R_CHARGE_Input
R_CHARGE_Output   --> C_STORE_Input   [positive capacitor lead]
C_STORE_Output    --> Battery_negative [negative capacitor lead]
```

Add the **measurement sense wires**:

```
LCD Terminal A    --> C_STORE_Input
LCD Terminal B    --> C_STORE_Output
```

The LCD is not a conducting return path. Do not omit the capacitor-to-battery-negative wire. Keep switch OPEN while constructing; close during Play Mode to charge.

5. Binding declarations and mappings

The creator predeclares these **runtime IDs** on `SparkLevelSceneBindings`:

```
Runtime Component IDs: R_CHARGE_Component, C_STORE_Component
Runtime Terminal IDs:  R_CHARGE_Input, R_CHARGE_Output,
                       C_STORE_Input, C_STORE_Output
```

The creator does **not** automatically map actual scene objects to these IDs.

If using a shelf workstation for R_CHARGE:

- Slot ID: `Register_Slot_CHARGE` (unique).
- Enable Smart Binding.
- Runtime Component ID: `R_CHARGE_Component`.
- Runtime Terminal A ID: `R_CHARGE_Input`.
- Runtime Terminal B ID: `R_CHARGE_Output`.
- Use `SparkLevelWorkstationBindingBridge` or your Binding Applier, then verify the slot and installed item resolve in Play Mode.

If using a fixed capacitor (recommended):

- In `SparkLevelSceneBindings`, map its `SparkElectricalComponent` and two `SparkTerminal` references using the **fixed component / fixed terminal binding tables**.
- Do **not** rely on declarations alone. If you map it as a fixed binding, move the capacitor IDs out of runtime-only declarations when required by your binding validation rules.
- Make sure `C_STORE_Component`, `C_STORE_Input`, and `C_STORE_Output` resolve at runtime.

If using a shelf-managed capacitor instead:

- Slot ID: `Capacitor_Slot_STORE` (unique); use the capacitor category supported by your shelf.
- Smart component ID: `C_STORE_Component`; terminal IDs: `C_STORE_Input` and `C_STORE_Output`.
- Validate its terminal binding support and workstation category before relying on the bridge.

Fixed source binding:

- Bind `PowerSupply_01` to the real 5 V source object.
- Bind `Battery_positive` and `Battery_negative` to the real source terminals.
- Check that the exact strings match `Level_21.asset` and that the level loader resolves them.
- Switch and LCD may also need their usual fixed component/terminal bindings depending on how your existing level prefabs configure them.

Quick binding audit (must all succeed): `PowerSupply_01`, `Battery_positive`, `Battery_negative`, `R_CHARGE_Component`, `R_CHARGE_Input`, `R_CHARGE_Output`, `C_STORE_Component`, `C_STORE_Input`, `C_STORE_Output`.

6. Set up level manager and tutorial

1. Add the `Level_21_Layout.prefab` and `Level_21.asset` pair to the existing `SparkLevelLayoutCatalog`, following the exact entry style of Level 20.
2. Add/register `Level_21.asset` in the level gameplay/progression list as appropriate, following Level 20.

3. In `SparkDrawCurrentLevel.drawings`, add `Level_21_Drawing.asset` (its `levelId` is `level_21`).
4. Verify the loader reports Level 21 as the active layout and that the whiteboard opens at Page 1 of 4.
5. Avoid modifying earlier level assets while registering this one.

7. Understand the three preliminary targets

Target ID	Measurement	Authored range
`level21_supply`	Supply voltage (Battery_positive minus Battery_negative)	4.8 to 5.2 V
`level21_capacitor_voltage`	Voltage across capacitor terminals	3.0 to 4.8 V
`level21_charge_current`	Magnitude of R_CHARGE current	0.15 to 2.2 mA

All three must pass **at the same time**. These ranges deliberately target the charging *transient*, not the final steady state. The authored target types are voltage range (5) and current range (6). Do not assume the generic evaluator samples at the correct time until verified.

For an ideal 5 V step, 1 kOhm series resistance, and 1000 uF capacitor: `tau = R*C = 1 second`.

After closing SW1	Ideal Vcap	Ideal resistor current
0.0 s	0.00 V	5.00 mA
1.0 s	3.16 V	1.84 mA
2.0 s	4.32 V	0.68 mA
3.0 s	4.75 V	0.25 mA
5.0 s	4.97 V	0.03 mA

These are **design reference values**, not guaranteed results from Project Spark's solver.

8. Test in this order (critical)

A — Bindings only

- Enter Play Mode on Level 21. Check active Level ID, source, component, and terminal binding audits. Fix unresolved targets before solving circuitry.

B — Battery and switch

- With SW1 open, capacitor should not charge. Confirm source terminal voltage is near 5 V.

C — Charge transient

- Start with capacitor uncharged; close SW1. Track Vcap and R_CHARGE current at ~0, 1, 2, 3, 5 seconds using solver telemetry.
- Vcap should rise monotonically toward ~5 V while current decreases toward zero.
- Open SW1: a disconnected ideal capacitor should approximately hold charge (actual decay depends on solver leakage/load model).
- Reset/restart level: confirm the capacitor actually returns to its intended initial state.

D — Completion conditions

- Check the three individual target result lines and overall target count.
- Expect targets to overlap roughly 1–3 seconds if solver time integration and evaluation update rate are suitable.
- If the solver jumps immediately to 5 V or never leaves 0 V, investigate capacitor stamping, timestep, history commits, state initialization, and real circuit connectivity.
- If voltage/current are correct but evaluation fails, inspect runtime bindings, `requireClosedReturn`, target evaluation cadence, and whether evaluator samples transient values. **Do not fake completion by lowering ranges arbitrarily.**

E — Regression

- Reopen and verify Level 18 (5/5), Level 19, and Level 20. Level 21 must not alter their binding declarations or solver behavior.

9. Common failures and precise fixes

Symptom	Likely cause	Check
Creator menu missing	Editor script not compiled / misplaced	`Editor` folder, Console errors, exact menu path
No active level	Catalog or progression entry missing	Level 21 catalog registration
`C_STORE` unresolved	Declared but no bound physical component	Fixed scene bindings or capacitor workstation bridge
`R_CHARGE` unresolved	Shelf slot not installed or wrong IDs	Slot ID, smart IDs, bridge diagnostics
Vcap always 0 V	Open wire/switch; solver not stamping capacitor	Circuit connections and capacitor solver registry
Vcap instantly 5 V	Steady-state-only solve or stale transient state	Timestep, integration and solver state update
No charging current	No conductive series path / wrong telemetry reference	R_CHARGE binding, switch and GND return
2/3 but no completion	Target windows don't coincide or transient missed	Per-target evaluator logs, update cadence
Completion triggers only after charge settles	Current/voltage target mode semantics	Inspect evaluator and choose genuine time-aware objective if needed
Earlier levels break	Shared asset/prefab was edited	Revert shared modifications; keep new work in Level 21

10. Ready-to-play acceptance checklist

- ☐ Unity compiles without errors.
- ☐ Level 21 opens through the normal level loader.
- ☐ Active source and every target terminal/component binding resolves.
- ☐ Physical wires form the exact series RC path and the LCD is a separate sense branch.
- ☐ Capacitor starts discharged, charges gradually, current decays naturally.
- ☐ All three targets pass for correct electrical reasons in Play Mode.
- ☐ Level drawing displays all four pages.
- ☐ No changes or regressions to Levels 1–20.

Important limitation: This guide is based on the generated Level 21 authoring package and available Project Spark architecture. It is not a claim that the scaffold is already wired or that the transient completion window has been tested in Unity.